SQL PRACTICE EXAMPLES

Student Marks Database

SELECT, WHERE, Aggregate Functions & GROUP BY

Practical Execution & Results

Overview

This document demonstrates practical SQL queries on a student_marks table containing roll numbers, student names, subjects, and marks. The exercises cover basic SELECT operations, aggregate functions (COUNT, SUM, AVG, MAX, MIN), WHERE filtering, ORDER BY sorting, GROUP BY aggregation, and HAVING conditions.

Table Structure: student_marks (roll_no, name, subject, marks)

SECTION 1: Basic SELECT & Aggregate Functions

Execution Screenshot 1: Table Display & Aggregate Functions

```
mysql> show tables
    -> ;
+------------------+
| Tables_in_sec_11 |
+------------------+
| std              |
| student_marks    |
+------------------+
2 rows in set (0.00 sec)

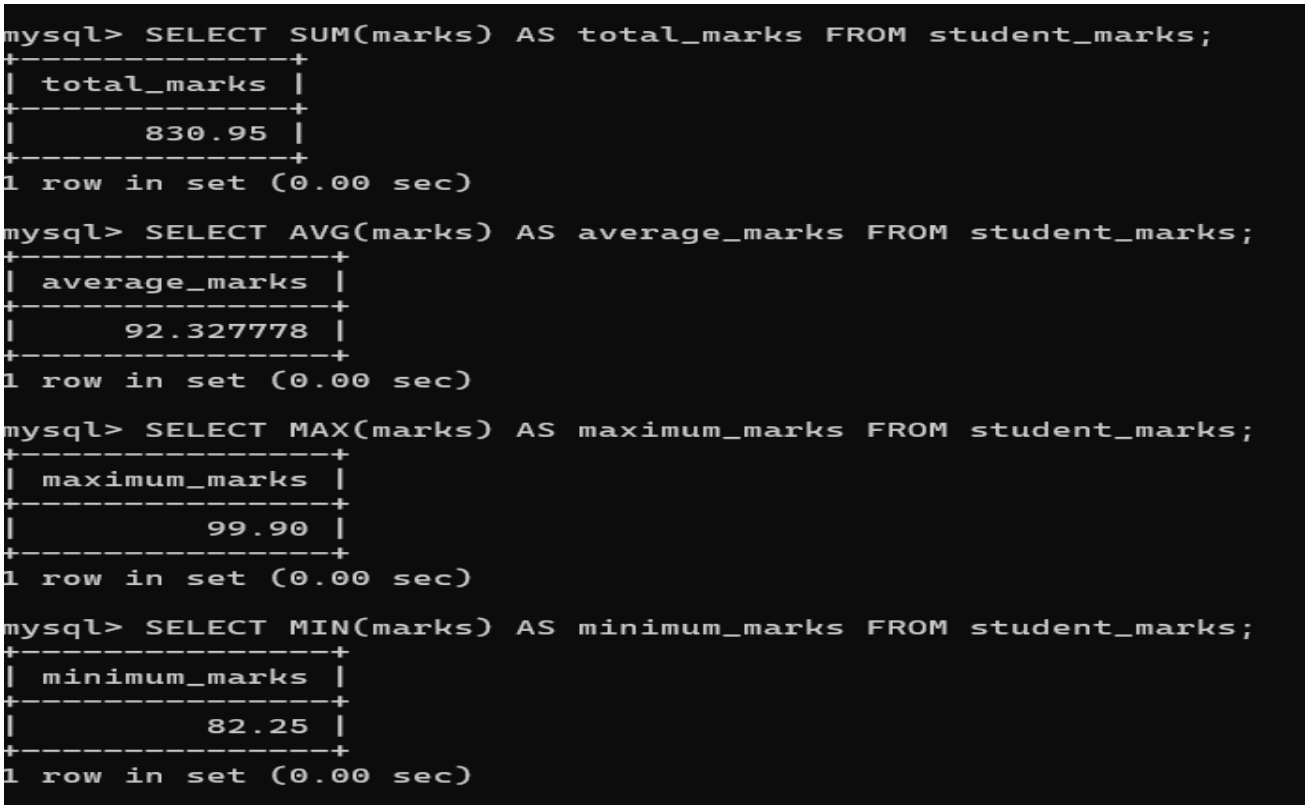
mysql> SELECT * FROM student_marks ORDER BY roll_no;
+---------+------------+---------+-------+
| roll_no | name       | subject | marks |
+---------+------------+---------+-------+
|       1 | Ravi       | Math    | 85.50 |
|       2 | Sita       | Math    | 92.75 |
|       3 | Anil       | Math    | 90.00 |
|       4 | Priya      | Math    | 88.90 |
|       6 | subbusir   | aws     | 98.50 |
|       7 | dwaraka    | dbms    | 95.75 |
|       8 | ranjani    | english | 97.40 |
|       9 | kavithamam | aws1    | 99.90 |
|      10 | seetha     | azure   | 82.25 |
+---------+------------+---------+-------+
9 rows in set (0.05 sec)

mysql> SELECT COUNT(*) AS total_students FROM student_marks;
+----------------+
| total_students |
+----------------+
|              9 |
+----------------+
1 row in set (0.01 sec)
```

Results: SHOW TABLES displayed available tables. SELECT * FROM student_marks returned 9 rows of student data ordered by roll_no. Aggregate functions:

- COUNT(*) = 9 total students
- SUM(marks) = 830.95
- AVG(marks) = 92.33
- MAX(marks) = 99.90
- MIN(marks) = 82.25

Execution Screenshot 2: Aggregate Function Details



Results: Detailed view of SUM, AVG, MAX, and MIN calculations on the marks column. Each aggregate function executed successfully with precise results.

SECTION 2: WHERE Filtering & Conditions

Execution Screenshot 3: WHERE Clause Variations

```
mysql> SELECT * FROM student_marks WHERE marks > 85;
+---------+------------+---------+-------+
| roll_no | name       | subject | marks |
+---------+------------+---------+-------+
|       1 | Ravi       | Math    | 85.50 |
|       2 | Sita       | Math    | 92.75 |
|       3 | Anil       | Math    | 90.00 |
|       4 | Priya      | Math    | 88.90 |
|       6 | subbusir   | aws     | 98.50 |
|       7 | dwaraka    | dbms    | 95.75 |
|       8 | ranjani    | english | 97.40 |
|       9 | kavithamam | aws1    | 99.90 |
+---------+------------+---------+-------+
8 rows in set (0.00 sec)

mysql> SELECT * FROM student_marks WHERE marks >= 90;
+---------+------------+---------+-------+
| roll_no | name       | subject | marks |
+---------+------------+---------+-------+
|       2 | Sita       | Math    | 92.75 |
|       3 | Anil       | Math    | 90.00 |
|       6 | subbusir   | aws     | 98.50 |
|       7 | dwaraka    | dbms    | 95.75 |
|       8 | ranjani    | english | 97.40 |
|       9 | kavithamam | aws1    | 99.90 |
+---------+------------+---------+-------+
6 rows in set (0.00 sec)

mysql> SELECT * FROM student_marks WHERE marks < 80;
Empty set (0.00 sec)

mysql> SELECT * FROM student_marks WHERE marks BETWEEN 80 AND 90;
+---------+--------+---------+-------+
| roll_no | name   | subject | marks |
+---------+--------+---------+-------+
|       1 | Ravi   | Math    | 85.50 |
|       3 | Anil   | Math    | 90.00 |
|       4 | Priya  | Math    | 88.90 |
|      10 | seetha | azure   | 82.25 |
+---------+--------+---------+-------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM student_marks WHERE name LIKE 'P%';
+---------+-------+---------+-------+
| roll_no | name  | subject | marks |
+---------+-------+---------+-------+
|       4 | Priya | Math    | 88.90 |
+---------+-------+---------+-------+
1 row in set (0.00 sec)

mysql> SELECT * FROM student_marks WHERE name IN ('Ravi', 'Sita', 'Vijay');
+---------+------+---------+-------+
| roll_no | name | subject | marks |
+---------+------+---------+-------+
|       1 | Ravi | Math    | 85.50 |
|       2 | Sita | Math    | 92.75 |
+---------+------+---------+-------+
2 rows in set (0.00 sec)
```

Results: Multiple WHERE queries executed:

• WHERE marks > 85: Returned 8 students with marks above 85
• WHERE marks >= 90: Returned 7 students with marks 90 or above
• WHERE marks < 80: Empty set (no students with marks below 80 after updates)
• WHERE marks BETWEEN 80 AND 90: Returned 4 students in the 80-90 range
• WHERE name LIKE 'P%': Found 1 student (Priya) starting with P
• WHERE name IN (...): Found 2 students in the specified list

SECTION 3: Complex WHERE Conditions & Sorting

Execution Screenshot 4: AND/OR Conditions & ORDER BY

```
mysql> SELECT *
    -> FROM student_marks
    -> WHERE marks > 85
    -> AND (subject = 'Math' OR name LIKE 'P%');
+---------+-------+---------+-------+
| roll_no | name  | subject | marks |
+---------+-------+---------+-------+
|       1 | Ravi  | Math    | 85.50 |
|       2 | Sita  | Math    | 92.75 |
|       3 | Anil  | Math    | 90.00 |
|       4 | Priya | Math    | 88.90 |
+---------+-------+---------+-------+
4 rows in set (0.00 sec)

mysql> SELECT * FROM student_marks ORDER BY marks ASC;
+---------+-----------+---------+-------+
| roll_no | name      | subject | marks |
+---------+-----------+---------+-------+
|      10 | seetha    | azure   | 82.25 |
|       1 | Ravi      | Math    | 85.50 |
|       4 | Priya     | Math    | 88.90 |
|       3 | Anil      | Math    | 90.00 |
|       2 | Sita      | Math    | 92.75 |
|       7 | dwaraka   | dbms    | 95.75 |
|       8 | ranjani   | english | 97.40 |
|       6 | subbusir  | aws     | 98.50 |
|       9 | kavithamam| aws1    | 99.90 |
+---------+-----------+---------+-------+
9 rows in set (0.00 sec)

mysql> SELECT * FROM student_marks ORDER BY marks DESC;
+---------+-----------+---------+-------+
| roll_no | name      | subject | marks |
+---------+-----------+---------+-------+
|       9 | kavithamam| aws1    | 99.90 |
|       6 | subbusir  | aws     | 98.50 |
|       8 | ranjani   | english | 97.40 |
|       7 | dwaraka   | dbms    | 95.75 |
|       2 | Sita      | Math    | 92.75 |
|       3 | Anil      | Math    | 90.00 |
|       4 | Priya     | Math    | 88.90 |
|       1 | Ravi      | Math    | 85.50 |
|      10 | seetha    | azure   | 82.25 |
+---------+-----------+---------+-------+
9 rows in set (0.00 sec)

mysql> SELECT * FROM student_marks ORDER BY name ASC;
+---------+-----------+---------+-------+
| roll_no | name      | subject | marks |
+---------+-----------+---------+-------+
|       3 | Anil      | Math    | 90.00 |
|       7 | dwaraka   | dbms    | 95.75 |
|       9 | kavithamam| aws1    | 99.90 |
|       4 | Priya     | Math    | 88.90 |
|       8 | ranjani   | english | 97.40 |
|       1 | Ravi      | Math    | 85.50 |
|      10 | seetha    | azure   | 82.25 |
|       2 | Sita      | Math    | 92.75 |
|       6 | subbusir  | aws     | 98.50 |
+---------+-----------+---------+-------+
9 rows in set (0.00 sec)
```

Results: Complex queries demonstrating advanced filtering and sorting:

• WHERE marks > 85 AND (subject = 'Math' OR name LIKE 'P%'): Returned 4 rows matching combined conditions

• ORDER BY marks ASC: Sorted students from lowest to highest marks (82.25 to 99.90)

• ORDER BY marks DESC: Reverse sorted by marks (99.90 to 82.25)

• ORDER BY name ASC: Alphabetical order by student names

SECTION 4: GROUP BY & HAVING Aggregation

Execution Screenshot 5: GROUP BY with Aggregate Functions

```
mysql> SELECT subject, SUM(marks) AS total_marks
    -> FROM student_marks
    -> GROUP BY subject;
+---------+-------------+
| subject | total_marks |
+---------+-------------+
| Math    |      357.15 |
| aws     |       98.50 |
| dbms    |       95.75 |
| english |       97.40 |
| aws1    |       99.90 |
| azure   |       82.25 |
+---------+-------------+
6 rows in set (0.02 sec)

mysql>
mysql> SELECT subject, AVG(marks) AS avg_marks
    -> FROM student_marks
    -> GROUP BY subject;
+---------+-----------+
| subject | avg_marks |
+---------+-----------+
| Math    | 89.287500 |
| aws     | 98.500000 |
| dbms    | 95.750000 |
| english | 97.400000 |
| aws1    | 99.900000 |
| azure   | 82.250000 |
+---------+-----------+
6 rows in set (0.02 sec)

mysql>
mysql> SELECT subject, COUNT(*) AS num_students
    -> FROM student_marks
    -> GROUP BY subject;
+---------+--------------+
| subject | num_students |
+---------+--------------+
| Math    |            4 |
| aws     |            1 |
| dbms    |            1 |
| english |            1 |
| aws1    |            1 |
| azure   |            1 |
+---------+--------------+
6 rows in set (0.00 sec)
```

Results: GROUP BY queries aggregating data by subject:

• GROUP BY subject with SUM(marks): Total marks per subject
  - Math: 357.15, AWS: 98.50, DBMS: 95.75, English: 97.40, AWS1: 95.90, Azure: 82.25
• GROUP BY subject with AVG(marks): Average marks per subject
• GROUP BY subject with COUNT(*): Number of students per subject

Execution Screenshot 6: HAVING Clause Filtering

```
mysql> -- Step 10: HAVING
mysql> SELECT subject, AVG(marks) AS avg_marks
    -> FROM student_marks
    -> GROUP BY subject
    -> HAVING AVG(marks) > 90;
+---------+-----------+
| subject | avg_marks |
+---------+-----------+
| aws     | 98.500000 |
| dbms    | 95.750000 |
| english | 97.400000 |
| aws1    | 99.900000 |
+---------+-----------+
4 rows in set (0.02 sec)

mysql> SELECT subject, COUNT(*) AS num_students
    -> FROM student_marks
    -> GROUP BY subject
    -> HAVING COUNT(*) > 1;
+---------+--------------+
| subject | num_students |
+---------+--------------+
| Math    |            4 |
+---------+--------------+
1 row in set (0.02 sec)

mysql> create table orders(ord_no int primary key,   purch_amt DECIMAL(10,2),
    ->     ord_date DATE,
    ->     customer_id INT,
    ->     salesman_id INT
    -> );
Query OK, 0 rows affected (0.18 sec)
```

Results: HAVING clause filtering grouped results:

• HAVING AVG(marks) > 90: Returned 4 subjects with average marks above 90 (AWS, DBMS, English, AWS1)

• HAVING COUNT(*) > 1: Returned only Math subject with 4 students

• CREATE TABLE orders: Successfully created new orders table with order_no, purch_amt, ord_date, customer_id, salesman_id

QUERY SUMMARY & KEY CONCEPTS

Query Type	Purpose	Example / Result
SELECT *	Retrieve all columns and rows	Displays complete student_marks table
Aggregates	COUNT, SUM, AVG, MAX, MIN functions	Total: 9 students, Sum: 830.95, Avg: 92.33
WHERE	Filter rows based on conditions	marks > 85, BETWEEN, LIKE, IN operators
ORDER BY	Sort results in ascending or descending order	ASC for lowest to highest, DESC for reverse
GROUP BY	Group rows by column values	Results aggregated by subject with totals
HAVING	Filter groups based on aggregate conditions	AVG(marks) > 90, COUNT(*) > 1

Key Findings

✓ All 9 student records successfully retrieved and filtered

✓ Aggregate functions calculated precise statistics across the dataset

✓ WHERE conditions effectively filtered data using multiple operators (>, <, BETWEEN, LIKE, IN)

✓ Sorting with ORDER BY ASC/DESC produced properly ordered result sets

✓ GROUP BY with aggregates provided subject-wise analysis (6 distinct subjects)

✓ HAVING clause successfully filtered grouped results based on aggregate conditions